Packaging reusable code

Module 5 - Lesson 3

Overview

Packaging turns your code into something installable and reusable. A pyproject.toml plus a standard layout lets others pip install your library.

Key Concepts

- Organize code into a package: a directory with __init__.py (or src layout).
- pyproject.toml declares metadata, dependencies, and build settings.
- Publish locally with pip install -e . to develop against your package.
- Upload to PyPI with build and twine for public distribution.
- Semantic versioning (major.minor.patch) communicates breaking changes.
- Document the public API so consumers know what is stable.

Example

```
# pyproject.toml
[project]
name = "mytoolbox"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = ["requests>=2.31"]

# install locally
$ pip install -e .
```

Summary

Packaging gives your code a name, version, and install command. A src layout, pyproject.toml, and versioning make libraries reusable across projects.

Practice Checklist

- Turn a script into a package with a src layout.
- Declare metadata and dependencies in pyproject.toml.
- Install it locally with pip install -e . and import it from another directory.